INGENIERÍA DE SOFTWARE I

# Entregable — Etapa 4: Modelo Entidad-Relación (ER)

*Proyecto NEXFIT*

## Datos generales del equipo

| GRUPO N° | FECHA DE ENTREGA |
|---|---|
| 1 | 14/10/2026 · Revisión: 16/10/2026 (v4) |

| Integrantes (nombre completo y código) |
|---|
| Andrés David Pérez Castañeda (202220379) |
| Javier Santiago Corredor Alvarez (202212259) |
| Jeferson Santiago Madera Gaona (202129186) |
| Santiago Munevar Espinel (202212446) |

Este modelo ER (v4) formaliza el entregable oficial de la Etapa 4 y está alineado con el análisis de dominio (Etapa 3) y el SRS v9. Parte del esquema PostgreSQL 16 que se ejecutó y probó el 12/10/2026 (75 pruebas) y agrega, según las decisiones del equipo: el motivo, la fecha y el autor del cambio de estado en memberships (D-05), la finalización de pagos pendientes en payments (D-04) y la caducidad de la contraseña temporal en users (D-15). Los cambios de la v2 están descritos en la migración Migracion_v3.sql, solo se probaron sobre tablas simuladas, y sus pruebas (T78 a T82) están por ejecutar. La v3 (16/10/2026) retira la tabla user_sessions: la sesión se guarda en el servidor con Spring Session JDBC (D-14, D-35, decisión de arquitectura TT-04), en sus tablas técnicas spring_session y spring_session_attributes, que crea la migración Flyway V1 del backend. La versión oficial es PostgreSQL 18 administrado en Neon (D-28): las pruebas T01 a T75 y T78 a T82 deben ejecutarse en ella. La EPS es obligatoria para los clientes (RN-25, decisión del docente del 14/10/2026); el esquema no cambia porque esa regla se valida en el backend. Los códigos D-xx remiten a la hoja «Decisiones y brechas» del Product Backlog (NexFit_Product_Backlog_Final.xlsx). La v4 (16/10/2026) incorpora los requisitos RF-113 a RF-118 del SRS v9, que vienen de los mockups del frontend aprobados por el equipo: categoría, nivel, duración y calorías aproximadas de la rutina (RF-113), entrenamiento realizado con sus ejercicios completados (RF-115, tablas workout_logs y workout_log_items), meta semanal (RF-117, columna en organization_members) y peso corporal (RF-118, tabla body_weight_records). La racha (RF-116) se calcula y no tiene tabla. El esquema pasa a 23 tablas y 10 tipos ENUM; los cambios v3 a v4 están en Migracion_v4.sql y sus pruebas T84 a T95 se ejecutaron solo sobre un esquema parcial.

# 1. Diagrama Entidad-Relación completo

El modelo integra 23 tablas agrupadas en **9 módulos**, con tipos de dato de PostgreSQL, claves primarias (PK), claves foráneas (FK), restricciones únicas (UQ) y cardinalidad en notación de pata de gallo en cada relación. Los diez tipos ENUM son: member_role, document_type, membership_status, payment_status, payment_method, class_status, booking_status, equipment_status, routine_category y routine_level (valores en la sección 7.1). La figura 1 muestra la vista general; la figura 2, el modelo completo con todas las columnas (para lectura en pantalla, ampliando); las figuras 3 a 7, el detalle por módulo.

*Fuente editable: archivo Model_Code_ER_NexFit_Group_1_v4.puml (PlantUML) y PNG independiente ER_NexFit_completo.png. Las figuras de este documento se generaron a partir de ese mismo archivo (23 tablas y 32 relaciones) con Graphviz en esta revisión. Puede abrirse en https://www.plantuml.com/plantuml/uml/ o con la extensión de PlantUML para VS Code.*

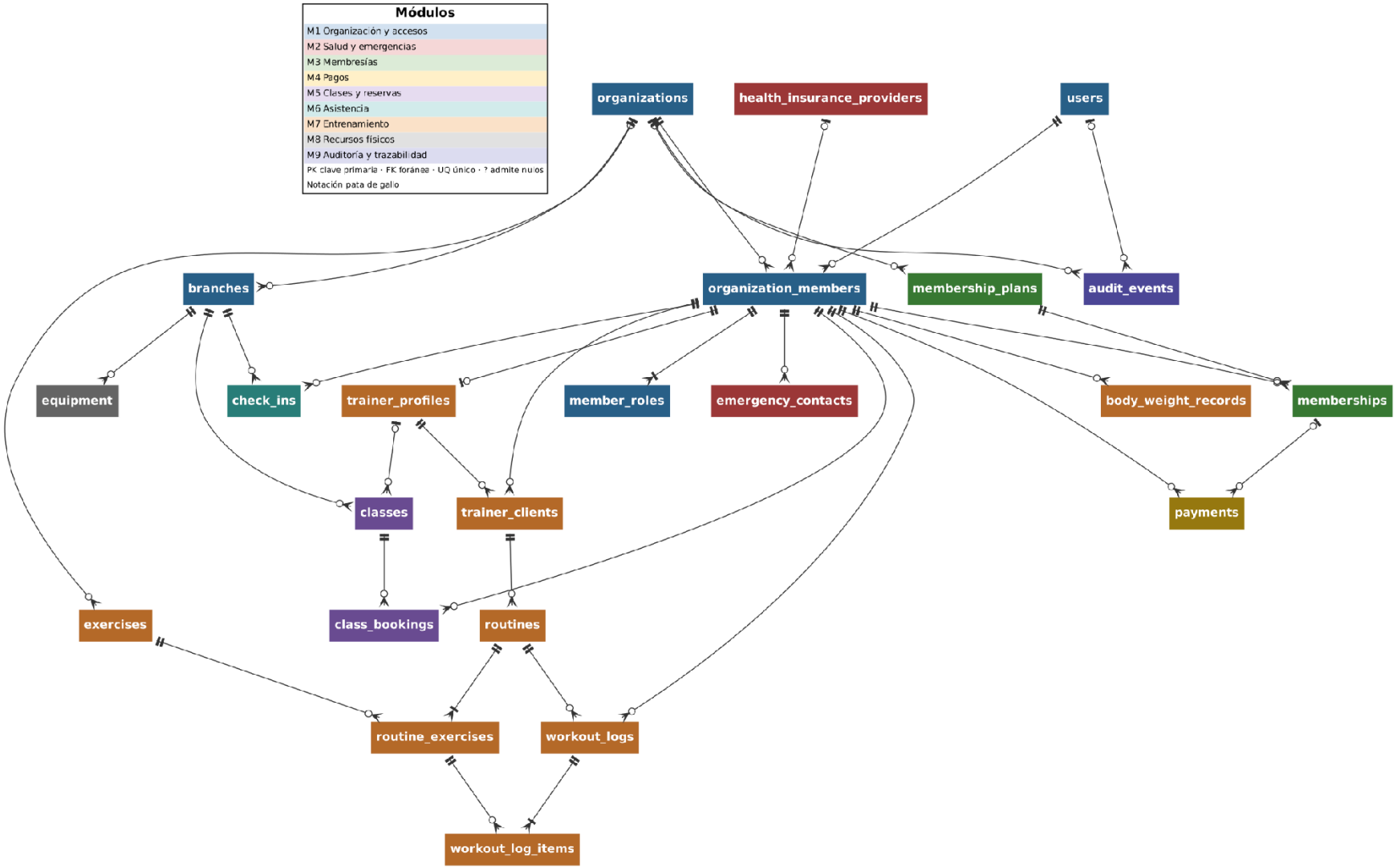



*Figura 1. Vista general: 23 tablas, 32 relaciones, colores por módulo.*

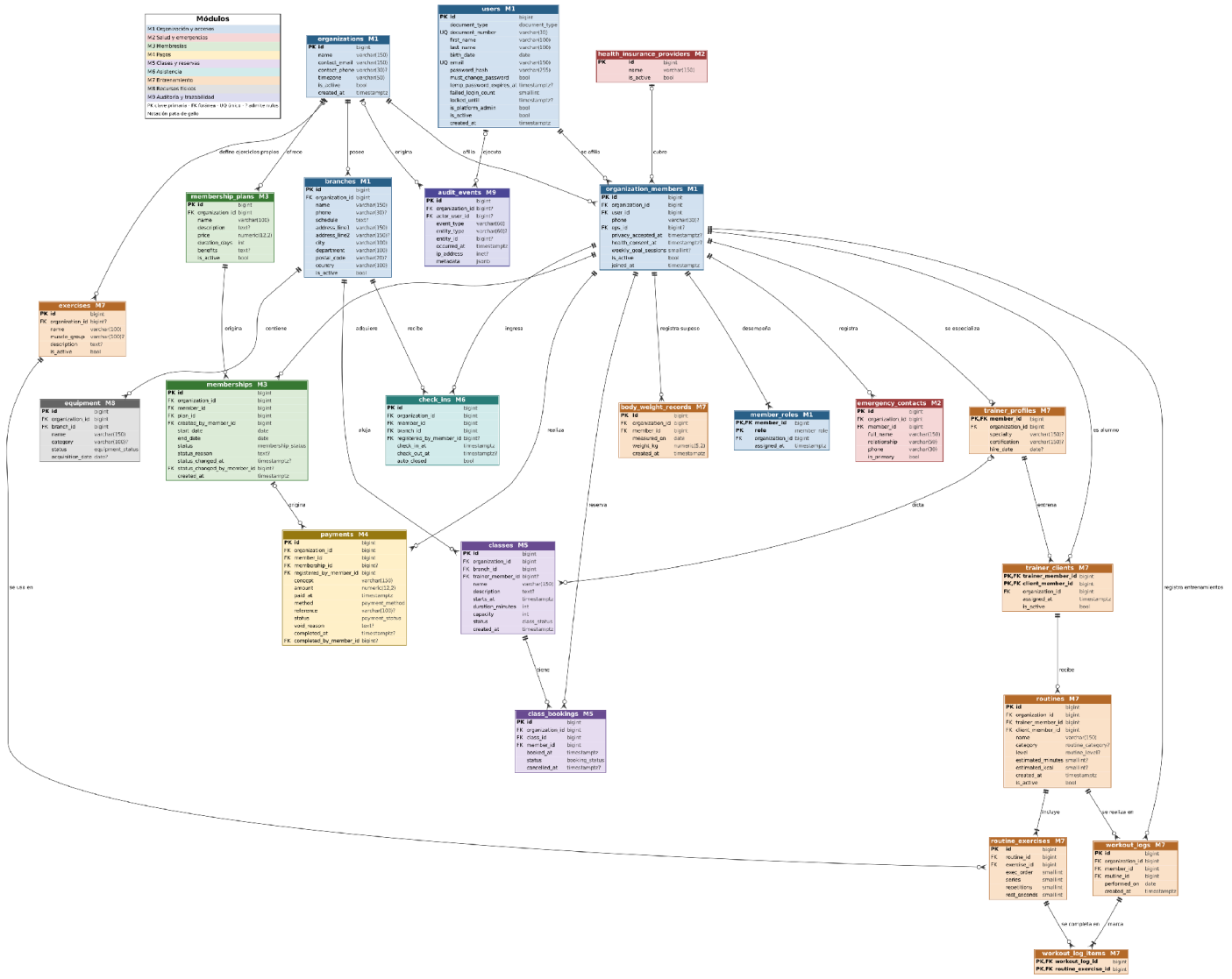



*Figura 2. Modelo ER completo con todas las columnas (ampliar para leer; PNG independiente adjunto).*

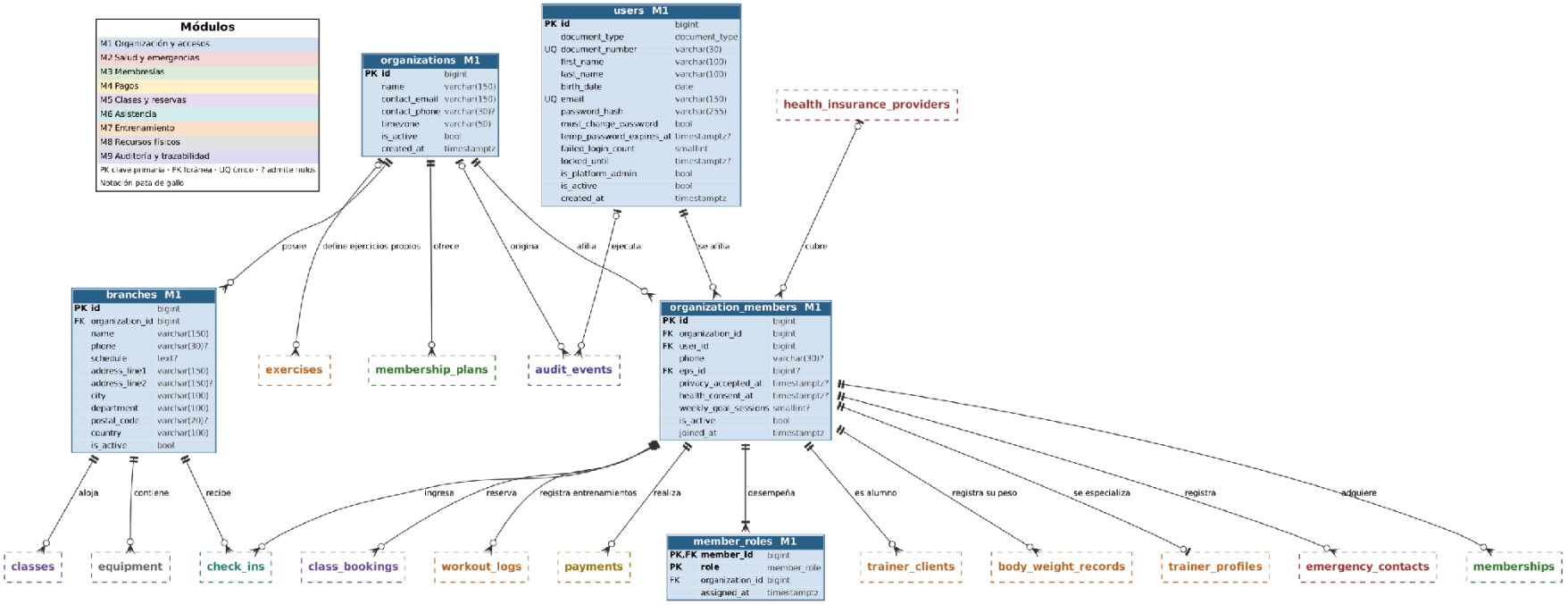



*Figura 3. Módulo M1 — Organización y accesos.*

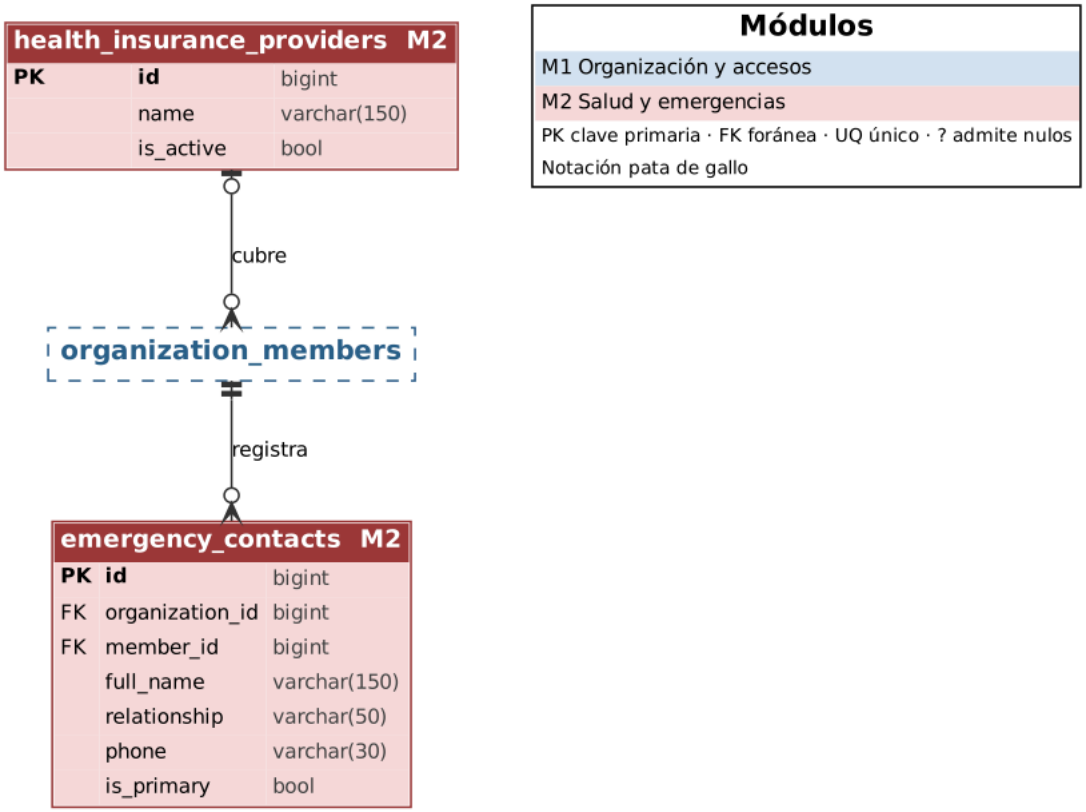



*Figura 4. Módulo M2 — Salud y emergencias.*

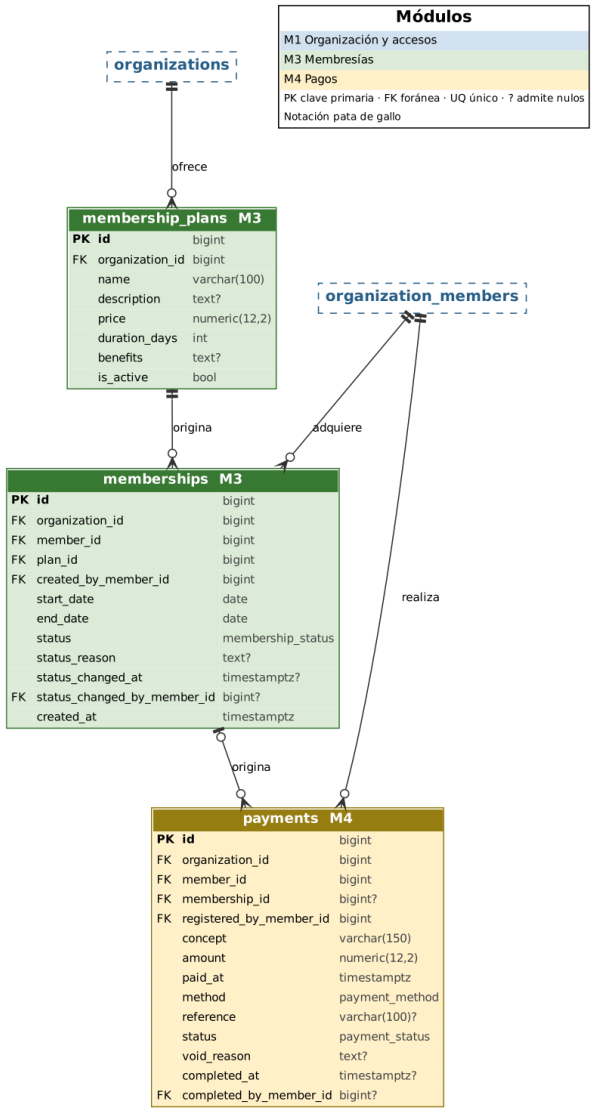



*Figura 5. Módulos M3 y M4 — Membresías y pagos (incluye las columnas añadidas en la v2).*

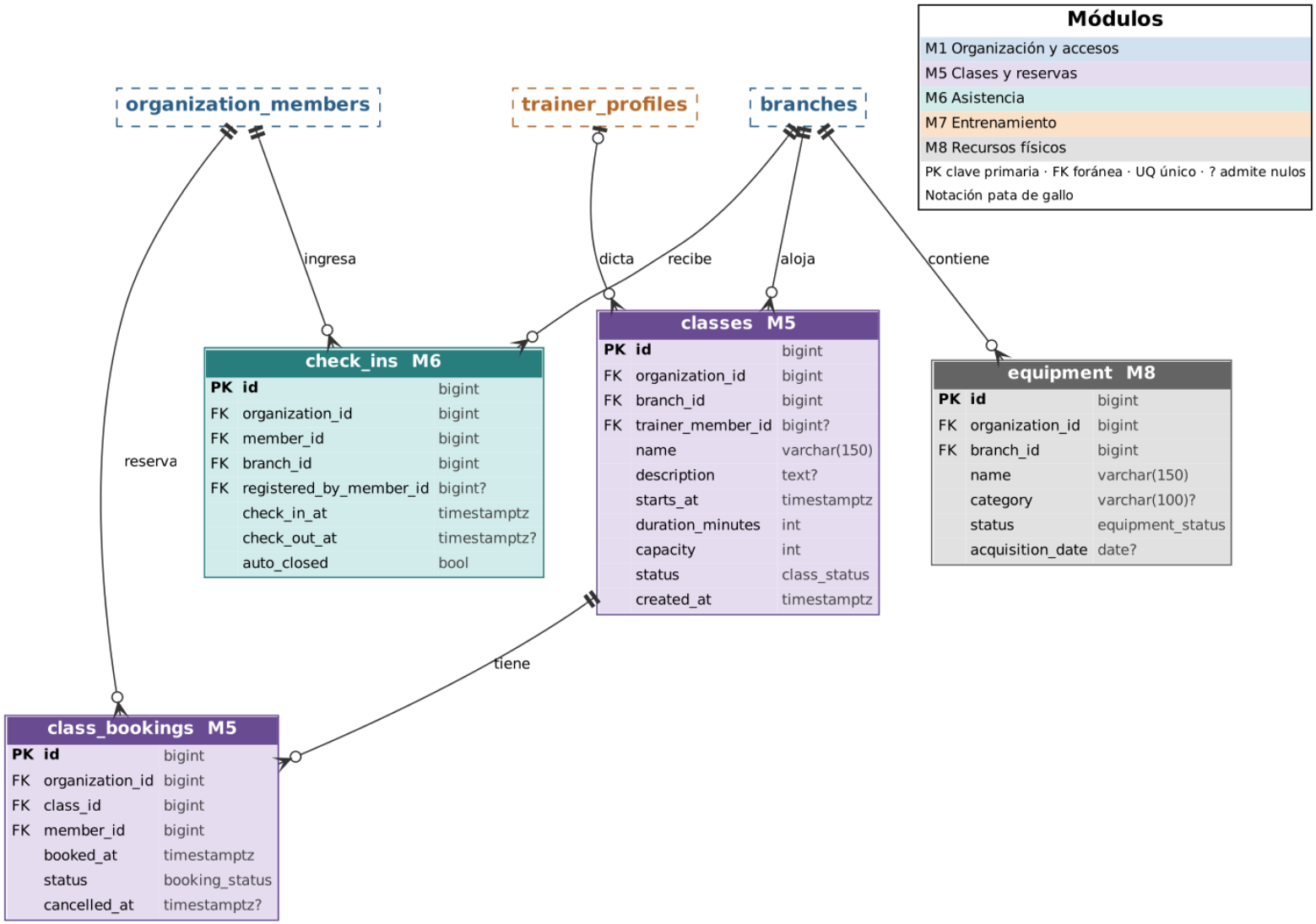



*Figura 6. Módulos M5, M6 y M8 — Clases y reservas, asistencia y equipo.*

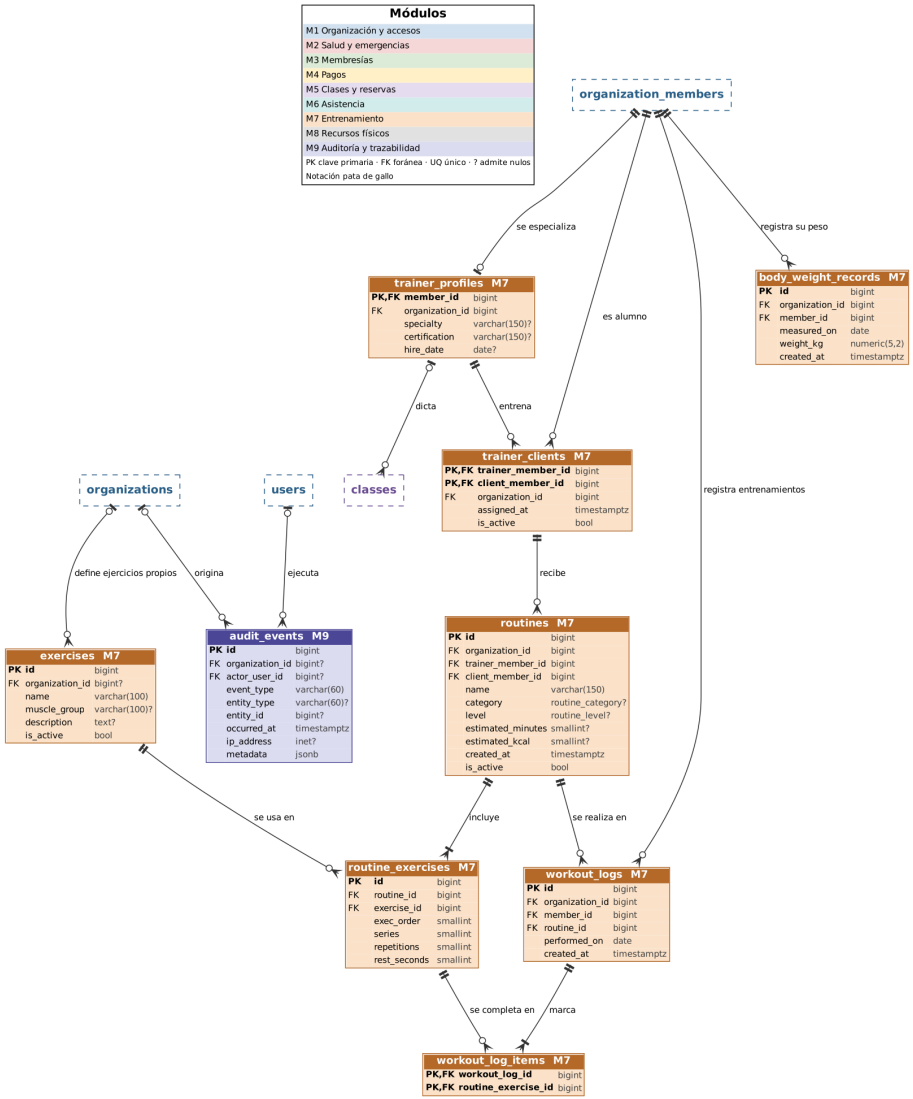



*Figura 7. Módulos M7 y M9 — Entrenamiento y auditoría.*

## 2. Trazabilidad con el modelo de dominio (Etapa 3)

| Entidad del modelo de dominio | Tabla en el modelo ER | Módulo | Observación |
|---|---|---|---|
| **Organización (Establecimiento / Gimnasio)** | organizations | M1 | is_active permite el borrado lógico; timezone (por defecto America/Bogota) define el «día» de cada establecimiento (RN-27). |
| **Sede** | branches | M1 | Dirección embebida (address_*); nombre único por gimnasio sin distinguir mayúsculas. Se elimina addresses y manager_member_id. |
| **Usuario** | users | M1 | Solo identidad y acceso (documento, nombre, correo, hash, bloqueo). Ya no tiene EPS, dirección ni teléfono. is_platform_admin marca al Administrador Master. temp_password_expires_at guarda la caducidad de la contraseña temporal (D-15). |
| **Afiliación** | organization_members | M1 | Une un usuario con un gimnasio y es el punto del que cuelga todo lo operativo. Guarda teléfono, EPS (obligatoria para clientes; nula para el personal) y las fechas de aceptación de la política y de autorización de datos de salud. Desde la v4 guarda además la meta semanal de entrenamientos del cliente (weekly_goal_sessions, RF-117). |
| **Sesión de usuario** | spring_session | M1 | Tabla técnica de Spring Session JDBC (no se dibuja): guarda la sesión en el servidor, su último acceso y su expiración a los 30 min de inactividad; el cierre de sesión la borra. El establecimiento activo es un atributo de la sesión (D-14, D-35). |
| **Rol en el establecimiento** | member_roles | M1 | Rol por afiliación (admin, staff, trainer, client); máximo un admin por gimnasio (índice único parcial). |
| **EPS** | health_insurance_providers | M2 | Catálogo global de EPS; nombre único; mantenido por el Administrador Master. |
| **Contacto de emergencia** | emergency_contacts | M2 | Pertenece a la afiliación (no al usuario); máximo un contacto principal por afiliación. |
| **Plan** | membership_plans | M3 | Nombre único por gimnasio; precio ≥ 0 y duración > 0 con CHECK. |
| **Membresía** | memberships | M3 | Plan y afiliación del mismo gimnasio por FK compuesta; end_date se guarda como fotografía; EXCLUDE impide períodos solapados no cancelados (RN-28). Se elimina la sede principal. status_reason, status_changed_at y status_changed_by_member_id guardan el motivo, la fecha y el autor de una suspensión o cancelación (D-05). |
| **Pago** | payments | M4 | membership_id opcional; la membresía debe ser del mismo cliente (FK compuesta). Estado voided con motivo; no se pueden borrar. completed_at y completed_by_member_id registran quién completó un pago pendiente y cuándo (D-04). |
| **Clase** | classes | M5 | trainer_member_id opcional (puede asignarse después); sede y entrenador del mismo gimnasio por FK compuesta. |
| **Reserva** | class_bookings | M5 | Unión N:M entre classes y afiliaciones cliente; estados confirmed y cancelled; índice único parcial por reserva confirmada (RN-16); trigger de cupo (RN-15). |
| **Asistencia** | check_ins | M6 | Registro manual (RN-18); registered_by_member_id nulo cuando lo registra el propio cliente; auto_closed marca cierres automáticos (RN-32); un solo ingreso abierto por cliente. |

| Entidad del modelo de dominio | Tabla en el modelo ER | Módulo | Observación |
|---|---|---|---|
| **Perfil de entrenador** | trainer_profiles | M7 | Especialización 1:1 opcional de la afiliación; solo existe si la afiliación tiene el rol trainer (trigger). |
| **Alumno (asignación entrenador–cliente)** | trainer_clients | M7 | Relación entrenador–cliente dentro de un mismo gimnasio; ambos roles se verifican por trigger. |
| **Ejercicio** | exercises | M7 | organization_id nulo = catálogo global; con valor = ejercicio propio del gimnasio. Nombre único por ámbito, sin distinguir mayúsculas. |
| **Rutina** | routines | M7 | Referencia a trainer_clients: solo se asigna a un alumno propio del entrenador (RN-17); is_active falso = archivada. Desde la v4 admite categoría, nivel, duración estimada y calorías aproximadas, todos opcionales (RF-113). |
| **Rutina_Ejercicio** | routine_exercises | M7 | Unión N:M entre routines y exercises (orden, series, repeticiones, descanso); trigger verifica que el ejercicio sea global o del mismo gimnasio. |
| **Entrenamiento realizado** | workout_logs | M7 | Registro de que un cliente realizó una de sus rutinas activas en una fecha local (RN-41); el trigger trg_workout_log_scope valida el alcance. La racha (RN-42) se calcula con estas filas y no se guarda. |
| **Ejercicio_Completado** | workout_log_items | M7 | Unión N:M entre workout_logs y routine_exercises: ejercicios de la rutina marcados en ese entrenamiento; trigger verifica que pertenezcan a la rutina. |
| **Registro de peso** | body_weight_records | M7 | Peso corporal en kg por cliente y día (20 a 500 kg); dato de salud visible solo para su titular (RN-43). |
| **Equipo** | equipment | M8 | Inventario por sede; estado con ENUM de cuatro valores. |
| **Evento de auditoría** | audit_events | M9 | Tabla de solo inserción (triggers impiden UPDATE y DELETE); organization_id nulo para eventos de plataforma. |

*Las 23 entidades de negocio del dominio corresponden a las 23 tablas; no hay tablas sin entidad ni entidades sin tabla. La racha es un concepto derivado: se calcula y no tiene tabla (RN-42). La entidad Sesión de usuario se guarda en las tablas técnicas de Spring Session (spring_session y spring_session_attributes), que crea la migración Flyway V1 del backend y no forman parte del diagrama (v3).*

## 3. Cardinalidad de las relaciones

| Entidad A | Entidad B | Cardinalidad | Descripción |
|---|---|---|---|
| users | organization_members | 1:N | Un usuario puede afiliarse a varios gimnasios; cada afiliación es de un solo usuario. |
| organizations | organization_members | 1:N | Un gimnasio afilia a muchos usuarios (a lo sumo una afiliación por usuario). |
| organization_members | member_roles | 1:N | Una afiliación tiene uno o varios roles. |
| organization_members | trainer_profiles | 1:1 (opcional) | Una afiliación puede extenderse a perfil de entrenador una sola vez. |
| organizations | branches | 1:N | Un gimnasio posee una o varias sedes; cada sede pertenece a un solo gimnasio (RN-21). |
| organizations | membership_plans | 1:N | Un gimnasio ofrece uno o varios planes (RN-09). |

| Entidad A | Entidad B | Cardinalidad | Descripción |
|---|---|---|---|
| organizations | exercises | 1:N (opcional) | Un ejercicio puede ser propio de un gimnasio; si no tiene gimnasio, es del catálogo global. |
| organizations | audit_events | 1:N (opcional) | Un evento pertenece a un gimnasio o a la plataforma (sin gimnasio). |
| users | audit_events | 1:N (opcional) | Un evento lo ejecuta un usuario; es nulo en intentos de acceso con usuario no identificado. |
| health_insurance_providers | organization_members | 1:N (opcional) | Una EPS cubre a muchas afiliaciones; la columna admite nulos porque el personal no registra EPS; para los clientes es obligatoria (se valida en el backend) y exige autorización registrada. |
| organization_members | emergency_contacts | 1:N | Una afiliación registra uno o varios contactos de emergencia (al menos uno para clientes, validado en la aplicación). |
| membership_plans | memberships | 1:N | Un plan origina muchas membresías. |
| organization_members | memberships | 1:N | Una afiliación cliente adquiere una o varias membresías a lo largo del tiempo (sin solaparse). |
| memberships | payments | 1:N (opcional) | Una membresía genera uno o varios pagos; un pago puede no ser de membresía. |
| organization_members | payments | 1:N | Una afiliación cliente realiza uno o varios pagos. |
| organization_members | check_ins | 1:N | Una afiliación cliente registra uno o varios ingresos. |
| branches | check_ins | 1:N | Una sede recibe uno o varios ingresos. |
| branches | classes | 1:N | Una sede aloja una o varias clases. |
| trainer_profiles | classes | 1:N (opcional) | Un entrenador dicta cero o varias clases; una clase puede no tener entrenador. |
| classes | class_bookings | 1:N | Una clase tiene cero o varias reservas. |
| organization_members | class_bookings | 1:N | Una afiliación cliente hace cero o varias reservas. |
| trainer_profiles | trainer_clients | 1:N | Un entrenador tiene cero o varios alumnos. |
| organization_members | trainer_clients | 1:N | Una afiliación cliente puede ser alumno de uno o varios entrenadores. |
| trainer_clients | routines | 1:N | Un alumno recibe cero o varias rutinas de su entrenador. |
| routines | routine_exercises | 1:N | Una rutina incluye uno o más ejercicios con sus parámetros. |
| exercises | routine_exercises | 1:N | Un ejercicio se usa en cero o varias rutinas. |
| organization_members | workout_logs | 1:N | Una afiliación cliente registra cero o varios entrenamientos realizados (RF-115). |
| routines | workout_logs | 1:N | Una rutina se realiza en cero o varios entrenamientos. |
| workout_logs | workout_log_items | 1:N | Un entrenamiento marca uno o varios ejercicios completados (al menos uno, se valida en el backend). |
| routine_exercises | workout_log_items | 1:N | Un ejercicio de rutina se completa en cero o varios entrenamientos. |

| Entidad A | Entidad B | Cardinalidad | Descripción |
|---|---|---|---|
| organization_members | body_weight_records | 1:N | Una afiliación cliente registra cero o varios pesos corporales, uno por día (RF-118). |
| branches | equipment | 1:N | Una sede contiene uno o varios equipos. |
| classes | organization_members | N:M | Se resuelve mediante class_bookings (Reserva). |
| trainer_profiles | organization_members (cliente) | N:M | Se resuelve mediante trainer_clients (Alumno). |
| routines | exercises | N:M | Se resuelve mediante routine_exercises. |
| workout_logs | routine_exercises | N:M | Se resuelve mediante workout_log_items. |

*Además de las 32 relaciones dibujadas, el esquema contiene claves foráneas compuestas adicionales hacia organization_members (por ejemplo, memberships.status_changed_by_member_id, payments.completed_by_member_id, memberships.created_by_member_id, payments.registered_by_member_id, check_ins.registered_by_member_id y classes.trainer_member_id → trainer_profiles). Comparten pareja de tablas con las relaciones listadas y se detallan en el diccionario de datos (sección 7).*

## 4. Tablas de unión para relaciones N:M

El modelo contiene cuatro relaciones muchos-a-muchos **entre entidades** (clases–clientes, rutinas–ejercicios, entrenadores–clientes, entrenamientos–ejercicios de la rutina), resueltas con una tabla de unión que lleva atributos propios, y **un atributo multivaluado** (los roles de una afiliación) resuelto también en una tabla. La tabla siguiente lista las cinco.

| Tabla de unión | Resuelve | Atributos propios | Restricción de unicidad |
|---|---|---|---|
| **class_bookings** | classes ↔ organization_members (cliente) | booked_at, status, cancelled_at | PK id; índice único parcial uq_bookings_one_active (class_id, member_id) cuando status = confirmed: permite reservar de nuevo tras cancelar (RN-16) |
| **routine_exercises** | routines ↔ exercises | exec_order, series, repetitions, rest_seconds | PK id; UNIQUE (routine_id, exec_order). Se usa clave sustituta porque un mismo ejercicio puede repetirse en la rutina |
| **trainer_clients** | trainer_profiles ↔ organization_members (cliente) | assigned_at, is_active | PK compuesta (trainer_member_id, client_member_id); CHECK entrenador distinto de cliente; trigger exige rol client |
| **member_roles** | organization_members ↔ member_role (valor enumerado) | assigned_at | PK compuesta (member_id, role); índice único parcial uq_one_admin_per_org. No es una N:M entre entidades sino un atributo multivaluado resuelto en tabla |
| **workout_log_items** | workout_logs ↔ routine_exercises | (ninguno) | PK compuesta (workout_log_id, routine_exercise_id): un ejercicio se marca una vez por entrenamiento; el trigger exige que sea de la rutina del entrenamiento |

# 5. Verificación de formas normales

## 5.1 Primera forma normal (1FN)

- Toda tabla tiene clave primaria (id sustituto o clave compuesta en member_roles, trainer_profiles y trainer_clients), sin filas duplicadas.
- Todos los atributos son atómicos. Los estados son ENUM, no listas de texto. membership_plans.benefits es un texto descriptivo; si se necesitara consultar beneficios individualmente, se extraería a benefits(plan_id, description). audit_events.metadata es JSONB de apoyo, no se usa para consultas de negocio.
- No hay grupos repetitivos: los contactos de emergencia están en emergency_contacts (una fila por contacto) y los roles en member_roles.

✔ **El modelo cumple 1FN.**

## 5.2 Segunda forma normal (2FN)

- Las tablas con clave simple no pueden tener dependencias parciales.
- En las tablas con clave compuesta (member_roles, trainer_profiles, trainer_clients) los atributos no clave dependen de la clave completa. En routine_exercises y class_bookings la unicidad de negocio se implementa sobre una clave sustituta.

✔ **El modelo cumple 2FN.**

## 5.3 Tercera forma normal (3FN) y redundancia deliberada de organization_id

- Ningún atributo no clave depende de otro atributo no clave dentro de su tabla. health_insurance_providers y los ENUM evitan repetir datos descriptivos.
- La identidad (users) está separada de la afiliación (organization_members): los datos de contacto, salud y consentimiento se guardan por gimnasio y los datos de identidad una sola vez.
- **Redundancia deliberada de organization_id.** Las tablas branches, organization_members, member_roles, emergency_contacts, membership_plans, memberships, payments, trainer_profiles, trainer_clients, classes, class_bookings, check_ins, routines, workout_logs, body_weight_records y equipment tienen organization_id (y exercises y audit_events, de forma opcional). En varias podría deducirse a través de otra tabla; se conserva porque cada clave foránea relevante es compuesta (organization_id, id): PostgreSQL rechaza cualquier fila que mezcle datos de dos gimnasios (por ejemplo, un plan de otro gimnasio en una membresía, o una sede ajena en un ingreso), de modo que no puede quedar inconsistente. Agregar organization_id a todas las tablas no basta: lo que protege es la clave compuesta que lo une al registro padre.
- **Tablas sin organization_id, por diseño:** organizations (es el propio gimnasio), users y health_insurance_providers (son globales), routine_exercises y workout_log_items (heredan el gimnasio de su rutina o de su entrenamiento; fn_routine_exercise_scope verifica que el ejercicio sea global o del mismo gimnasio, y fn_workout_log_item_scope que pertenezca a la rutina del entrenamiento). exercises tiene organization_id opcional (nulo = catálogo global) y audit_events también (nulo = evento de plataforma).

### Otras decisiones documentadas

- memberships.end_date podría derivarse de start_date + duración del plan, pero se guarda como fotografía del acuerdo: si el plan cambia, las membresías contratadas no se alteran (RN-29). El trigger fn_check_membership verifica que sea coherente al insertar.
- payments.member_id es alcanzable también vía membership_id, pero se conserva porque hay pagos sin membresía; la clave compuesta (membership_id, member_id) impide que apunten a clientes distintos.

- El estado efectivo de la membresía (programada, vigente, vencida, suspendida, cancelada) se calcula en la vista v_memberships_effective a partir de las fechas y la zona horaria del establecimiento; no depende de un proceso que actualice una columna.
- Se eliminó la tabla addresses: la dirección es un atributo de la sede y no se comparte, evitando alias mutables entre usuarios y sedes.
- El motivo y la fecha de una suspensión o cancelación se guardan en la propia membresía y no solo en audit_events.metadata, para consultarlos sin interpretar JSON; la auditoría conserva además el evento (D-05). La sesión no se modela como tabla de negocio: la guarda Spring Session en el servidor y el cierre de sesión la elimina (D-14, D-35). La unicidad de nombre en sedes, planes y ejercicios es por gimnasio (índices con lower(name)), no global.
- La racha no se almacena: se calcula en el backend con las fechas de workout_logs (RN-42), así que no puede quedar desactualizada ni depende de un proceso programado. La meta semanal es una columna de organization_members (weekly_goal_sessions) porque es un único valor por afiliación; el peso corporal tiene su propia tabla porque es una serie histórica (RF-118).
- body_weight_records guarda un dato de salud: solo lo consulta el propio cliente (RN-43, RNF-08) y, como el resto de datos de desarrollo y pruebas, debe ser ficticio.

✔ **El modelo cumple 3FN.**

# 6. Reglas del SRS v9 y cómo las garantiza el modelo

*«BD» = garantizada por restricciones o triggers y probada en la sección 8; «Backend» = regla de permisos o de proceso que se valida en el servicio de aplicación (no se pudo probar en la base de datos).*

| Regla (SRS v9) | Enunciado | Dónde | Mecanismo | Pruebas ejecutadas |
|---|---|---|---|---|
| RN-01 | Cada persona tendrá una única identidad (usuario) en NEXFIT y podrá estar afiliada a uno o más establecimientos, con a lo sumo una afiliación por establecimiento. | BD | UNIQUE (document_type, document_number); uq_users_email; UNIQUE (organization_id, user_id) | T01, T02, T03, T04 |
| RN-02 | La información operativa de una persona como cliente (membresías, pagos, asistencias, reservas, rutinas) y sus datos de contacto, EPS y contactos de emergencia serán independientes en cada establecimiento al que pertenezca. | BD | Datos operativos y de contacto cuelgan de organization_members; FK compuestas (organization_id, id) | T04, T12 |
| RN-03 | Un usuario solamente podrá acceder a información de un establecimiento en el contexto de los roles que tenga en él; sus permisos son la unión de esos roles y opera en un único establecimiento activo por sesión. Un usuario puede tener varios roles en el mismo establecimiento (por ejemplo, Administrador y Staff); el rol de Administrador no concede por sí mismo permisos operativos. | Backend | Middleware de autorización con establecimiento y roles tomados de la sesión (RF-04, RF-101) | — |
| RN-04 | Cada establecimiento tendrá a lo sumo un Administrador y deberá tener uno asignado para operar. | BD | Índice único parcial uq_one_admin_per_org | T05 |
| RN-05 | El Administrador Master podrá asignar o reemplazar el Administrador de un establecimiento; el reemplazo retira el rol al anterior en la misma transacción. | Backend | Transacción de reemplazo de administrador (RF-11) | — |
| RN-06 | El Staff y el Entrenador tendrán permisos diferenciados de acuerdo con la matriz de permisos de la sección 3.7. | Backend | Matriz de permisos 3.7 aplicada por endpoint | — |
| RN-07 | El Staff será responsable de registrar nuevos clientes; los Entrenadores no podrán crear clientes. | Backend | Permiso de creación de clientes solo para el rol Staff | — |
| RN-08 | El Administrador Master podrá gestionar información administrativa de los establecimientos y ver cifras agregadas, pero no podrá consultar ni modificar la información personal, membresías, pagos, asistencias, rutinas, EPS o contactos de los clientes finales. | Backend | El Master no tiene endpoints de datos de cliente; solo agregados | — |
| RN-09 | Cada establecimiento podrá definir y administrar sus propios planes de membresía de forma independiente; el nombre de un plan es único dentro del establecimiento sin distinguir mayúsculas. | BD | uq_plans_name (organization_id, lower(name)) | T13 |
| RN-10 | Una membresía deberá estar asociada a un único cliente (afiliación con rol Cliente) y a un plan activo pertenecientes al mismo establecimiento. | BD | FK compuesta (organization_id, plan_id); trigger fn_check_membership (rol Cliente, plan activo) | T14, T15, T17 |
| RN-11 | Un cliente podrá tener membresías independientes en diferentes establecimientos. | BD | La membresía referencia la afiliación, no la persona | T04 |
| RN-12 | Cada clase deberá pertenecer a un único establecimiento y estar asociada a una sede perteneciente al mismo establecimiento. | BD | FK compuesta (organization_id, branch_id) en classes | T39 |

| Regla (SRS v9) | Enunciado | Dónde | Mecanismo | Pruebas ejecutadas |
|---|---|---|---|---|
| RN-13 | Un cliente podrá registrar asistencia únicamente en una sede activa perteneciente a un establecimiento en el que tenga una membresía vigente (RN-27). | BD | Trigger fn_check_check_in sobre check_ins | T30, T31, T33 |
| RN-14 | Una reserva solo podrá realizarse sobre una clase programada que no haya iniciado, cuando el cliente tenga una membresía vigente el día de la clase en ese establecimiento y exista cupo disponible. | BD | Trigger fn_check_booking sobre class_bookings | T50, T51, T52, T53 |
| RN-15 | Una clase no podrá superar su capacidad máxima mediante reservas confirmadas, ni siquiera con solicitudes simultáneas. | BD | Trigger con SELECT … FOR UPDATE sobre la clase | T44, T45, T48, Concurrencia |
| RN-16 | Un cliente no podrá mantener más de una reserva confirmada para la misma clase; después de cancelar podrá reservar de nuevo si hay cupo. | BD | Índice único parcial uq_bookings_one_active | T43, T49 |
| RN-17 | Las rutinas solo podrán ser creadas y asignadas por un Entrenador autorizado, para alumnos activos suyos, dentro del establecimiento correspondiente. | BD + Backend | FK compuesta routines → trainer_clients; alumno activo se valida en backend | T61, T62 |
| RN-18 | El registro de asistencia se realizará de forma manual por el propio cliente o por Staff autorizado, y se conserva quién lo registró. | BD | check_ins.registered_by_member_id (nulo = el propio cliente) | T31 |
| RN-19 | El MVP almacenará registros de pagos, pero no implementará una pasarela de pagos real para procesar transacciones electrónicas. | Backend | No se implementa integración de pagos (decisión de alcance) | — |
| RN-20 | La información de clientes, membresías, pagos, asistencias, reservas y rutinas deberá mantenerse asociada al establecimiento correspondiente. | BD | organization_id en tablas de negocio con FK compuestas | T07, T12, T25, T34, T39, T40, T60, T65, T70 |
| RN-21 | Una sede solo podrá pertenecer a un establecimiento y no podrá utilizarse para registrar operaciones de otro establecimiento. | BD | FK compuestas (organization_id, branch_id); trigger de sede activa | T33, T34, T70 |
| RN-22 | Un entrenador solamente podrá gestionar alumnos, clases y rutinas dentro de los establecimientos en los que tenga el rol de entrenador. | BD | trainer_profiles + triggers fn_trainer_profile_requires_role y fn_keep_trainer_role | T55, T56, T40 |
| RN-24 | Una rutina solamente podrá ser consultada por el entrenador autorizado correspondiente y por el cliente al que haya sido asignada, dentro del establecimiento; ni el Administrador ni el Staff ni el Master la consultan. | Backend | Autorización por entrenador asignado o cliente propietario | — |
| RN-25 | Al registrar un cliente se le exigen una EPS del catálogo y al menos un contacto de emergencia; ambos son obligatorios por decisión del docente. | Backend + BD | El backend exige EPS y ≥ 1 contacto al afiliar a un cliente (eps_id admite nulos porque el personal no registra EPS); CHECK ck_eps_requires_consent | T08, T09, T10 |
| RN-26 | Cada persona se identifica de forma única en NEXFIT por su tipo y número de documento y, de forma independiente, su correo electrónico no puede repetirse (sin distinguir mayúsculas). | BD | UNIQUE (document_type, document_number) y uq_users_email | T01, T02 |
| RN-27 | Una membresía está vigente un día D, según la zona horaria del establecimiento, si su estado es «activa» y su fecha de inicio ≤ D ≤ su fecha de vencimiento. | BD | Función fn_has_valid_membership y vista v_memberships_effective | T52, T74, T75 |
| RN-28 | Un cliente no podrá tener dos membresías no canceladas con períodos que se solapen; una renovación inicia el día siguiente al vencimiento de la vigente. | BD | EXCLUDE ex_memberships_no_overlap (btree_gist) | T19, T20, T22 |
| RN-29 | El vencimiento de una membresía es inicio + duración del plan − 1 día y se conserva como fotografía: modificar o desactivar el plan no altera las membresías ya contratadas. | BD | Trigger fn_check_membership (end_date = start_date + duration − 1) | T16 |

| Regla (SRS v9) | Enunciado | Dónde | Mecanismo | Pruebas ejecutadas |
|---|---|---|---|---|
| RN-30 | Una membresía suspendida o cancelada no da acceso ni permite reservar; al suspenderla o cancelarla se cancelan las reservas futuras del cliente; la suspensión no extiende el vencimiento y la cancelación es definitiva. El motivo, la fecha y el autor del cambio de estado se guardan en la membresía. | Backend + BD | CHECK ck_memberships_status_reason: exige motivo, fecha y autor cuando el estado no es activo; el backend cancela las reservas futuras en la misma transacción. | — |
| RN-31 | Los pagos no se eliminan: un pago erróneo se anula indicando el motivo; anular un pago no modifica automáticamente la membresía asociada. | BD | Trigger trg_payments_no_delete; CHECK de motivo de anulación; un pago pendiente se completa una sola vez registrando completed_at y completed_by_member_id (ck_payments_completed_pair) | T26, T27, T28, T29 |
| RN-32 | Un cliente tiene a lo sumo un ingreso abierto; una tarea programada cierra, al final de cada día local del establecimiento, los ingresos que siguen abiertos y los marca como cierre automático (salida estimada, no una salida real). | BD + Backend | Tarea programada que cierra los ingresos abiertos al final del día local del establecimiento y los marca como cierre automático (salida estimada); el trigger fn_check_check_in se conserva como segunda protección; uq_checkins_one_open | T32, T35, T36, T37 |
| RN-33 | Un establecimiento inactivo no permite operar a ninguno de sus usuarios; una sede inactiva no recibe ingresos ni clases nuevas; un plan inactivo no puede contratarse; una afiliación inactiva no puede iniciar sesión en ese establecimiento. | Backend + BD | Backend bloquea operación por estado; BD rechaza ingresos y contratación de planes inactivos | T15, T33 |
| RN-34 | Los datos de identidad (documento, nombre, fecha de nacimiento, correo) solo los modifica su titular; los datos de contacto, EPS y contactos de emergencia los modifican el Staff y el propio cliente en su afiliación. Al buscar un documento existente, el Staff solo ve nombre y apellido. La corrección de un dato de identidad solo es posible antes de que el titular active la cuenta, con auditoría; después se atiende por solicitud verificable. | Backend | Endpoints de búsqueda devuelven solo nombre y apellido; edición por campo según rol | — |
| RN-35 | En el MVP solo se registran como clientes o personal personas mayores de 18 años. | Backend | Validación de fecha de nacimiento en el alta de usuario | — |
| RN-36 | Una clase no puede reducir su capacidad por debajo de sus reservas confirmadas; un entrenador no puede tener dos clases activas con horarios solapados; cancelar una clase cancela sus reservas confirmadas. | Backend | Validaciones de capacidad, solapamiento de entrenador y cancelación en cascada dentro de transacción | — |
| RN-37 | El tratamiento de datos personales requiere registrar la aceptación de la política de tratamiento; la EPS (dato de salud) solo se guarda con autorización expresa registrada; como la EPS es obligatoria para los clientes (RN-25), esa autorización también lo es al afiliarlos. | BD + Backend | CHECK ck_eps_requires_consent; aceptación de política y autorización de salud obligatorias al afiliar a un cliente | T08, T09 |
| RN-38 | Las cuentas creadas por terceros nacen con contraseña temporal que debe cambiarse en el primer ingreso; tras 5 intentos fallidos la cuenta se bloquea 15 minutos; el restablecimiento asistido lo hace el Staff o el Administrador solo para cuentas con una única afiliación (la suya) o sin ingreso previo, y el Master para cualquier cuenta. La contraseña temporal caduca; el cambio voluntario de contraseña exige la contraseña actual. | Backend + BD | Columnas must_change_password, failed_login_count y locked_until; reglas en el servicio de autenticación; temp_password_expires_at (caducidad de la temporal); la sesión se guarda en el servidor (Spring Session) y puede invalidarse | — |
| RN-39 | El catálogo de ejercicios global lo mantiene el Administrador Master; cada establecimiento puede crear ejercicios propios, visibles solo para él y para sus rutinas; el nombre es único dentro de su ámbito. | BD | uq_exercises_name (COALESCE(organization_id,0), lower(name)); trigger fn_routine_exercise_scope | T63, T64, T65, T67, T68 |

| Regla (SRS v9) | Enunciado | Dónde | Mecanismo | Pruebas ejecutadas |
|---|---|---|---|---|
| RN-40 | Los eventos de auditoría y los pagos no se modifican ni se eliminan una vez registrados (los pagos solo cambian de estado a «anulado»). | BD | Triggers de solo inserción en audit_events y de no borrado en payments | T28, T72, T73 |
| RN-41 | Un entrenamiento solo puede registrarse para una rutina activa asignada al propio cliente, lo registra él mismo, con la fecha local del establecimiento y sin fecha futura. | BD + Backend | Triggers trg_workout_log_scope (rutina del cliente y del gimnasio, activa, fecha no futura) y trg_workout_log_item_scope (ejercicio de la rutina); el backend registra solo para el cliente de la sesión | T84 a T90 |
| RN-42 | La racha cuenta los días consecutivos con al menos un entrenamiento registrado; se mantiene si el último fue hoy o ayer y, si no, vale 0; solo cuenta los del mismo establecimiento. | Backend | Consulta sobre workout_logs (organization_id, member_id, performed_on) con la fecha local del establecimiento; el valor no se almacena | — |
| RN-43 | Los entrenamientos, la meta semanal y el peso corporal solo los consulta el propio cliente en su establecimiento (el peso es un dato de salud, RNF-08); no los ven Staff, Administrador, Master ni Entrenador. El peso va de 20 a 500 kg, uno por día; un segundo registro del mismo día reemplaza al anterior. | Backend + BD | Endpoints solo del propio cliente; CHECK de peso (20 a 500 kg) y de meta (1 a 7); UNIQUE uq_body_weight_member_day (el reemplazo del día lo hace el servicio) | T91 a T94 |

*Las reglas con mecanismo «Backend» (RN-03, RN-05 a RN-08, RN-19, RN-24, RN-30, RN-34 a RN-36, RN-42 y RN-43) son reglas de permisos, de proceso o de formulario que no se expresan solo con restricciones de tabla; cada una tiene al menos una prueba automática prevista (RNF-05).*

# 7. Esquema físico y diccionario de datos

## 7.1 Tipos enumerados

| Tipo | Valores |
|---|---|
| **member_role** | admin, staff, trainer, client |
| **document_type** | CC, CE, PPT, PA |
| **membership_status** | active, suspended, cancelled |
| **payment_status** | pending, completed, voided |
| **payment_method** | cash, card, transfer, other |
| **class_status** | scheduled, cancelled |
| **booking_status** | confirmed, cancelled |
| **equipment_status** | available, maintenance, out_of_service, retired |
| **routine_category** | strength, cardio, mobility |
| **routine_level** | beginner, intermediate, advanced |

## 7.2 Tablas

*Notación: «?» = admite nulos; claves PK/FK/UQ según el esquema. Las claves foráneas compuestas (organization_id, id) aparecen en la lista de restricciones de cada tabla.*

### M1 · Organización y accesos

**organizations**

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| name | varchar(150) | NO | | |
| contact_email | varchar(150) | NO | | |
| contact_phone | varchar(30) | SÍ | | |
| timezone | varchar(50) | NO | | 'America/Bogota' |
| is_active | bool | NO | | true |
| created_at | timestamptz | NO | | now() |

**branches**

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| name | varchar(150) | NO | UQ por gimnasio | |

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| phone | varchar(30) | SÍ | | |
| schedule | text | SÍ | | |
| address_line1 | varchar(150) | NO | | |
| address_line2 | varchar(150) | SÍ | | |
| city | varchar(100) | NO | | |
| department | varchar(100) | NO | | |
| postal_code | varchar(20) | SÍ | | |
| country | varchar(100) | NO | | 'Colombia' |
| is_active | bool | NO | | true |

**Restricciones e índices:**

- FOREIGN KEY (organization_id) REFERENCES organizations(id)
- ÍNDICE ÚNICO uq_branches_name ON branches USING btree (organization_id, lower((name)::text))

### users

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| document_type | document_type | NO | | |
| document_number | varchar(30) | NO | UQ | |
| first_name | varchar(100) | NO | | |
| last_name | varchar(100) | NO | | |
| birth_date | date | NO | | |
| email | varchar(150) | NO | UQ | |
| password_hash | varchar(255) | NO | | |
| must_change_password | bool | NO | | true |
| temp_password_expires_at | timestamptz | SÍ | | |
| failed_login_count | smallint | NO | | 0 |
| locked_until | timestamptz | SÍ | | |
| is_platform_admin | bool | NO | | false |
| is_active | bool | NO | | true |
| created_at | timestamptz | NO | | now() |

**Restricciones e índices:**

- CHECK ((birth_date >= '1900-01-01'::date))
- CHECK ((failed_login_count >= 0))
- UNIQUE (document_type, document_number)
- ÍNDICE ÚNICO uq_users_email ON users USING btree (lower((email)::text))

### organization_members

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| user_id | bigint | NO | FK | |
| phone | varchar(30) | SÍ | | |
| eps_id | bigint | SÍ | FK | |
| privacy_accepted_at | timestamptz | SÍ | | |
| health_consent_at | timestamptz | SÍ | | |
| weekly_goal_sessions | smallint | SÍ | | |
| is_active | bool | NO | | true |
| joined_at | timestamptz | NO | | now() |

**Restricciones e índices:**

- CHECK (weekly_goal_sessions IS NULL OR weekly_goal_sessions BETWEEN 1 AND 7)
- CHECK (((eps_id IS NULL) OR (health_consent_at IS NOT NULL)))
- FOREIGN KEY (eps_id) REFERENCES health_insurance_providers(id)
- FOREIGN KEY (organization_id) REFERENCES organizations(id)
- FOREIGN KEY (user_id) REFERENCES users(id)

### member_roles

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| member_id | bigint | NO | PK FK | |
| role | member_role | NO | PK | |
| organization_id | bigint | NO | FK | |
| assigned_at | timestamptz | NO | | now() |

**Restricciones e índices:**

- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)
- UNIQUE (organization_id, member_id, role)
- ÍNDICE ÚNICO uq_one_admin_per_org ON member_roles USING btree (organization_id) WHERE (role = 'admin'::member_role)

## M2 · Salud y emergencias

### health_insurance_providers

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| name | varchar(150) | NO | UQ | |
| is_active | bool | NO | | true |

**Restricciones e índices:**

- ÍNDICE ÚNICO uq_eps_name ON health_insurance_providers USING btree (lower((name)::text))

### emergency_contacts

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| member_id | bigint | NO | FK | |
| full_name | varchar(150) | NO | | |
| relationship | varchar(50) | NO | | |
| phone | varchar(30) | NO | | |
| is_primary | bool | NO | | false |

**Restricciones e índices:**

- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)
- ÍNDICE ÚNICO uq_one_primary_contact ON emergency_contacts USING btree (member_id) WHERE is_primary

## M3 · Membresías

### membership_plans

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| name | varchar(100) | NO | UQ por gimnasio | |
| description | text | SÍ | | |
| price | numeric(12,2) | NO | | |
| duration_days | int | NO | | |
| benefits | text | SÍ | | |
| is_active | bool | NO | | true |

**Restricciones e índices:**

- CHECK ((duration_days > 0))
- CHECK ((price >= (0)::numeric))
- FOREIGN KEY (organization_id) REFERENCES organizations(id)
- ÍNDICE ÚNICO uq_plans_name ON membership_plans USING btree (organization_id, lower((name)::text))

### memberships

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| member_id | bigint | NO | FK | |
| plan_id | bigint | NO | FK | |
| created_by_member_id | bigint | NO | FK | |
| start_date | date | NO | | |
| end_date | date | NO | | |
| status | membership_status | NO | | 'active'::membership_status |
| status_reason | text | SÍ | | |
| status_changed_at | timestamptz | SÍ | | |
| status_changed_by_member_id | bigint | SÍ | FK | |
| created_at | timestamptz | NO | | now() |

**Restricciones e índices:**

- CHECK ((end_date >= start_date))
- FOREIGN KEY (organization_id, created_by_member_id) REFERENCES organization_members(organization_id, id)
- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)
- FOREIGN KEY (organization_id, plan_id) REFERENCES membership_plans(organization_id, id)
- UNIQUE (id, member_id)
- EXCLUDE USING gist (member_id WITH =, daterange(start_date, end_date, '[]'::text) WITH &&) WHERE ((status <> 'cancelled'::membership_status))
- FOREIGN KEY (organization_id, status_changed_by_member_id) REFERENCES organization_members(organization_id, id)
- CHECK (((status = 'active'::membership_status) OR ((status_reason IS NOT NULL) AND (status_changed_at IS NOT NULL) AND (status_changed_by_member_id IS NOT NULL)))) (ck_memberships_status_reason)

## M4 · Pagos

### payments

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| member_id | bigint | NO | FK | |

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| membership_id | bigint | SÍ | FK | |
| registered_by_member_id | bigint | NO | FK | |
| concept | varchar(150) | NO | | |
| amount | numeric(12,2) | NO | | |
| paid_at | timestamptz | NO | | now() |
| method | payment_method | NO | | |
| reference | varchar(100) | SÍ | | |
| status | payment_status | NO | | 'completed'::payment_status |
| void_reason | text | SÍ | | |
| completed_at | timestamptz | SÍ | | |
| completed_by_member_id | bigint | SÍ | FK | |

**Restricciones e índices:**

- CHECK ((amount > (0)::numeric))
- CHECK (((status <> 'voided'::payment_status) OR (void_reason IS NOT NULL)))
- FOREIGN KEY (membership_id, member_id) REFERENCES memberships(id, member_id)
- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)
- FOREIGN KEY (organization_id, registered_by_member_id) REFERENCES organization_members(organization_id, id)
- FOREIGN KEY (organization_id, completed_by_member_id) REFERENCES organization_members(organization_id, id)
- CHECK (((completed_at IS NULL) = (completed_by_member_id IS NULL))) (ck_payments_completed_pair)

## M5 · Clases y reservas

### classes

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| branch_id | bigint | NO | FK | |
| trainer_member_id | bigint | SÍ | FK | |
| name | varchar(150) | NO | | |
| description | text | SÍ | | |
| starts_at | timestamptz | NO | | |
| duration_minutes | int | NO | | |
| capacity | int | NO | | |
| status | class_status | NO | | 'scheduled'::class_status |

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| created_at | timestamptz | NO | | now() |

**Restricciones e índices:**

- CHECK ((capacity > 0))
- CHECK (((duration_minutes >= 1) AND (duration_minutes <= 480)))
- FOREIGN KEY (organization_id, branch_id) REFERENCES branches(organization_id, id)
- FOREIGN KEY (organization_id, trainer_member_id) REFERENCES trainer_profiles(organization_id, member_id)

### class_bookings

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| class_id | bigint | NO | FK | |
| member_id | bigint | NO | FK | |
| booked_at | timestamptz | NO | | now() |
| status | booking_status | NO | | 'confirmed'::booking_status |
| cancelled_at | timestamptz | SÍ | | |

**Restricciones e índices:**

- CHECK (((status = 'cancelled'::booking_status) = (cancelled_at IS NOT NULL)))
- FOREIGN KEY (organization_id, class_id) REFERENCES classes(organization_id, id)
- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)
- ÍNDICE ÚNICO uq_bookings_one_active ON class_bookings USING btree (class_id, member_id) WHERE (status = 'confirmed'::booking_status)

## M6 · Asistencia

### check_ins

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| member_id | bigint | NO | FK | |
| branch_id | bigint | NO | FK | |
| registered_by_member_id | bigint | SÍ | FK | |
| check_in_at | timestamptz | NO | | now() |
| check_out_at | timestamptz | SÍ | | |
| auto_closed | bool | NO | | false |

**Restricciones e índices:**

- CHECK (((check_out_at IS NULL) OR (check_out_at >= check_in_at)))
- FOREIGN KEY (organization_id, branch_id) REFERENCES branches(organization_id, id)
- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)
- FOREIGN KEY (organization_id, registered_by_member_id) REFERENCES organization_members(organization_id, id)
- ÍNDICE ÚNICO uq_checkins_one_open ON check_ins USING btree (member_id) WHERE (check_out_at IS NULL)

## M7 · Entrenamiento

### trainer_profiles

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| member_id | bigint | NO | PK FK | |
| organization_id | bigint | NO | FK | |
| specialty | varchar(150) | SÍ | | |
| certification | varchar(150) | SÍ | | |
| hire_date | date | SÍ | | |

**Restricciones e índices:**

- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)

### trainer_clients

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| trainer_member_id | bigint | NO | PK FK | |
| client_member_id | bigint | NO | PK FK | |
| organization_id | bigint | NO | FK | |
| assigned_at | timestamptz | NO | | now() |
| is_active | bool | NO | | true |

**Restricciones e índices:**

- CHECK ((trainer_member_id <> client_member_id))
- FOREIGN KEY (organization_id, client_member_id) REFERENCES organization_members(organization_id, id)
- FOREIGN KEY (organization_id, trainer_member_id) REFERENCES trainer_profiles(organization_id, member_id)
- UNIQUE (organization_id, trainer_member_id, client_member_id)

### exercises

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | SÍ | FK | |

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| name | varchar(100) | NO | UQ por gimnasio | |
| muscle_group | varchar(100) | SÍ | | |
| description | text | SÍ | | |
| is_active | bool | NO | | true |

**Restricciones e índices:**

- FOREIGN KEY (organization_id) REFERENCES organizations(id)
- ÍNDICE ÚNICO uq_exercises_name ON exercises USING btree (COALESCE(organization_id, (0)::bigint), lower((name)::text))

### routines

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| trainer_member_id | bigint | NO | FK | |
| client_member_id | bigint | NO | FK | |
| name | varchar(150) | NO | | |
| category | routine_category | SÍ | | |
| level | routine_level | SÍ | | |
| estimated_minutes | smallint | SÍ | | |
| estimated_kcal | smallint | SÍ | | |
| created_at | timestamptz | NO | | now() |
| is_active | bool | NO | | true |

**Restricciones e índices:**

- CHECK (estimated_minutes IS NULL OR estimated_minutes > 0)
- CHECK (estimated_kcal IS NULL OR estimated_kcal > 0)
- FOREIGN KEY (organization_id, trainer_member_id, client_member_id) REFERENCES trainer_clients(organization_id, trainer_member_id, client_member_id)

### routine_exercises

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| routine_id | bigint | NO | FK | |
| exercise_id | bigint | NO | FK | |
| exec_order | smallint | NO | | |

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| series | smallint | NO | | |
| repetitions | smallint | NO | | |
| rest_seconds | smallint | NO | | 0 |

**Restricciones e índices:**

- CHECK ((exec_order > 0))
- CHECK ((repetitions > 0))
- CHECK ((rest_seconds >= 0))
- CHECK ((series > 0))
- FOREIGN KEY (exercise_id) REFERENCES exercises(id)
- FOREIGN KEY (routine_id) REFERENCES routines(id)
- UNIQUE (routine_id, exec_order)

### workout_logs

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| member_id | bigint | NO | FK | |
| routine_id | bigint | NO | FK | |
| performed_on | date | NO | | |
| created_at | timestamptz | NO | | now() |

**Restricciones e índices:**

- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)
- FOREIGN KEY (routine_id) REFERENCES routines(id)
- ÍNDICE idx_workout_logs_member_day ON workout_logs (organization_id, member_id, performed_on)

### workout_log_items

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| workout_log_id | bigint | NO | PK FK | |
| routine_exercise_id | bigint | NO | PK FK | |

**Restricciones e índices:**

- PRIMARY KEY (workout_log_id, routine_exercise_id)
- FOREIGN KEY (routine_exercise_id) REFERENCES routine_exercises(id)
- FOREIGN KEY (workout_log_id) REFERENCES workout_logs(id)

### body_weight_records

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| member_id | bigint | NO | FK | |
| measured_on | date | NO | | |
| weight_kg | numeric(5,2) | NO | | |
| created_at | timestamptz | NO | | now() |

**Restricciones e índices:**

- CHECK ((weight_kg >= 20) AND (weight_kg <= 500))
- FOREIGN KEY (organization_id, member_id) REFERENCES organization_members(organization_id, id)
- UNIQUE (organization_id, member_id, measured_on)

## M8 · Recursos físicos

### equipment

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | NO | FK | |
| branch_id | bigint | NO | FK | |
| name | varchar(150) | NO | | |
| category | varchar(100) | SÍ | | |
| status | equipment_status | NO | | 'available'::equipment_status |
| acquisition_date | date | SÍ | | |

**Restricciones e índices:**

- FOREIGN KEY (organization_id, branch_id) REFERENCES branches(organization_id, id)

## M9 · Auditoría y trazabilidad

### audit_events

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| id | bigint | NO | PK | identidad |
| organization_id | bigint | SÍ | FK | |
| actor_user_id | bigint | SÍ | FK | |
| event_type | varchar(60) | NO | | |

| Columna | Tipo | Nulo | Clave | Valor por defecto |
|---|---|---|---|---|
| entity_type | varchar(60) | SÍ | | |
| entity_id | bigint | SÍ | | |
| occurred_at | timestamptz | NO | | now() |
| ip_address | inet | SÍ | | |
| metadata | jsonb | NO | | '{}' |

**Restricciones e índices:**

- FOREIGN KEY (actor_user_id) REFERENCES users(id)
- FOREIGN KEY (organization_id) REFERENCES organizations(id)

## 7.3 Vista, funciones y triggers

**Vista v_memberships_effective**: expone cada membresía con su estado efectivo (programada, vigente, vencida, suspendida, cancelada) calculado con la fecha local del establecimiento.

| Función | Responsabilidad |
|---|---|
| **fn_member_has_role(miembro, rol)** | Devuelve si una afiliación tiene un rol; la usan los triggers de entrenador, alumno y membresía. |
| **fn_has_valid_membership(org, miembro, fecha)** | Implementa la vigencia de RN-27 (estado activa y fecha dentro del período); la usan los triggers de ingreso y de reserva. |
| **fn_check_membership()** | Valida que la afiliación tenga rol cliente, que el plan esté activo y que end_date = start_date + duración − 1 (RN-10, RN-29). |
| **fn_payment_client()** | Valida que el pago sea de una afiliación cliente y de la misma afiliación de la membresía. |
| **fn_check_check_in()** | Valida sede activa, membresía vigente y salida posterior al ingreso; cierra automáticamente ingresos abiertos de días anteriores (RN-13, RN-32). |
| **fn_check_booking()** | Bloquea la fila de la clase (SELECT … FOR UPDATE) y valida clase programada y no iniciada, membresía vigente el día de la clase y cupo (RN-14, RN-15). |
| **fn_trainer_profile_requires_role(), fn_keep_trainer_role()** | Mantienen la coherencia entre el rol trainer y el perfil de entrenador. |
| **fn_trainer_client_roles()** | Exige rol trainer en el entrenador y rol client en el alumno de trainer_clients. |
| **fn_routine_exercise_scope()** | Exige que el ejercicio de una rutina sea global o del mismo gimnasio que la rutina (RN-39). |
| **fn_workout_log_scope()** | Exige que la rutina del entrenamiento sea del cliente y del gimnasio del registro, esté activa y que la fecha no sea futura según la zona horaria del establecimiento (RN-41). |
| **fn_workout_log_item_scope()** | Exige que el ejercicio marcado pertenezca a la rutina del entrenamiento (RN-41). |
| **fn_forbid_delete(), fn_forbid_update()** | Impiden borrar pagos y modificar o borrar eventos de auditoría (RN-31, RN-40). |

| Tabla | Trigger |
|---|---|
| audit_events | trg_audit_no_delete |
| audit_events | trg_audit_no_update |

| Tabla | Trigger |
|---|---|
| check_ins | trg_check_check_in |
| class_bookings | trg_check_booking |
| member_roles | trg_keep_trainer_role |
| memberships | trg_check_membership |
| payments | trg_payment_client |
| payments | trg_payments_no_delete |
| routine_exercises | trg_routine_exercise_scope |
| trainer_clients | trg_trainer_client_roles |
| trainer_profiles | trg_trainer_profile_role |
| workout_log_items | trg_workout_log_item_scope |
| workout_logs | trg_workout_log_scope |

# 8. Pruebas ejecutadas sobre PostgreSQL 16

Se ejecutó el esquema completo en PostgreSQL 16 y un conjunto de **75 pruebas** (archivo nexfit_tests.sql) con datos ficticios de dos gimnasios: **75 de 75** dieron el resultado esperado. Cada prueba intenta una operación que debe rechazarse (por ejemplo, un cruce entre gimnasios) o una que debe aceptarse, y compara con el resultado esperado. Durante la primera ejecución fallaron cuatro pruebas por errores del propio script de pruebas (datos de apoyo y números de identidad fijos), no del esquema; se corrigieron y se repitió la ejecución completa.

Alcance de la versión 2: las 75 pruebas T01 a T75 se ejecutaron sobre el esquema de la versión anterior. Las pruebas T78 a T82 cubren los cambios de la v2 y están por escribir y ejecutar; T76, T77 y T83 probaban user_sessions y se retiran en la v3. Hasta ejecutarlas no puede afirmarse «80 de 80». La versión oficial es PostgreSQL 18 en Neon (D-28): deben repetirse todas en ella (tarea TT-05 del Product Backlog).

Alcance de la versión 4: las pruebas T84 a T95 cubren los requisitos RF-113 a RF-118. Se ejecutaron el 16/10/2026 en PostgreSQL 16 con Migracion_v4.sql sobre un esquema parcial (solo las tablas que la v4 referencia: organizations, organization_members, routines, routine_exercises y exercises) y las 12 dieron el resultado esperado. Deben repetirse sobre el esquema completo en PostgreSQL 18 (TT-05).

| N° | Prueba | Resultado esperado | Resultado |
|---|---|---|---|
| T01 | documento duplicado | Rechazo | Correcto |
| T02 | correo duplicado sin distinguir mayúsculas | Rechazo | Correcto |
| T03 | afiliación duplicada en el mismo gimnasio | Rechazo | Correcto |
| T04 | una persona con afiliaciones independientes en 2 gimnasios | Según condición | Correcto |
| T05 | segundo administrador en el mismo gimnasio | Rechazo | Correcto |
| T06 | un miembro con varios roles (staff + trainer) | Aceptación | Correcto |
| T07 | rol de afiliación con organización incorrecta | Rechazo | Correcto |
| T08 | EPS sin autorización de datos de salud | Rechazo | Correcto |
| T09 | EPS con autorización registrada | Aceptación | Correcto |
| T10 | contacto de emergencia principal | Aceptación | Correcto |
| T11 | segundo contacto principal | Rechazo | Correcto |

| N° | Prueba | Resultado esperado | Resultado |
|---|---|---|---|
| T12 | contacto con organización que no es la de la afiliación | Rechazo | Correcto |
| T13 | plan repetido en el mismo gimnasio (sin distinguir mayúsculas) | Rechazo | Correcto |
| T14 | membresía con plan de otro gimnasio (RN-10) | Rechazo | Correcto |
| T15 | membresía con plan desactivado | Rechazo | Correcto |
| T16 | vencimiento incoherente con la duración del plan | Rechazo | Correcto |
| T17 | membresía para afiliación que no es cliente | Rechazo | Correcto |
| T18 | contratación válida (cliente 4) | Aceptación | Correcto |
| T19 | membresía solapada del mismo cliente | Rechazo | Correcto |
| T20 | renovación contigua (inicia al día siguiente del vencimiento) | Aceptación | Correcto |
| T21 | membresías del cliente 5 y 6 | Aceptación | Correcto |
| T22 | membresía cancelada no bloquea una nueva solapada | Aceptación | Correcto |
| T23 | pago asociado a la membresía del mismo cliente | Aceptación | Correcto |
| T24 | pago cuya membresía es de otro cliente | Rechazo | Correcto |
| T25 | pago de un cliente de otro gimnasio | Rechazo | Correcto |
| T26 | pago anulado sin motivo | Rechazo | Correcto |
| T27 | pago anulado con motivo | Aceptación | Correcto |
| T28 | los pagos no se pueden borrar | Rechazo | Correcto |
| T29 | pago con valor cero o negativo | Rechazo | Correcto |
| T30 | ingreso sin membresía vigente (RN-13) | Rechazo | Correcto |
| T31 | ingreso con membresía vigente (cliente 4, sede 1) | Aceptación | Correcto |
| T32 | segundo ingreso abierto el mismo día | Rechazo | Correcto |
| T33 | ingreso en sede inactiva (RN-21) | Rechazo | Correcto |
| T34 | ingreso en sede de otro gimnasio (RN-21) | Rechazo | Correcto |
| T35 | ingreso olvidado de ayer (cliente 5) | Aceptación | Correcto |
| T36 | nuevo ingreso hoy cierra automáticamente el de ayer | Aceptación | Correcto |
| T37 | el ingreso de ayer quedó cerrado y marcado auto_closed | Según condición | Correcto |
| T38 | salida anterior al ingreso | Rechazo | Correcto |
| T39 | clase con sede de otro gimnasio | Rechazo | Correcto |
| T40 | clase con entrenador de otro gimnasio | Rechazo | Correcto |
| T41 | clase con capacidad cero | Rechazo | Correcto |
| T42 | reserva válida (cliente 4) | Aceptación | Correcto |
| T43 | reserva confirmada duplicada (RN-16) | Rechazo | Correcto |
| T44 | segunda reserva (cliente 5) llena la clase | Aceptación | Correcto |
| T45 | sobrecupo (RN-15) | Rechazo | Correcto |
| T46 | cancelación antes del inicio | Aceptación | Correcto |
| T47 | el cupo liberado se puede reservar | Aceptación | Correcto |
| T48 | cliente que canceló no recupera cupo si la clase se llenó | Rechazo | Correcto |

| N° | Prueba | Resultado esperado | Resultado |
|---|---|---|---|
| T49 | reservar de nuevo tras cancelar cuando hay cupo | Aceptación | Correcto |
| T50 | reserva en clase cancelada | Rechazo | Correcto |
| T51 | reserva en clase ya iniciada | Rechazo | Correcto |
| T52 | reserva cuando la membresía vence antes de la clase | Rechazo | Correcto |
| T53 | reserva de cliente sin membresía (RN-14) | Rechazo | Correcto |
| T54 | estado cancelado sin fecha de cancelación | Rechazo | Correcto |
| T55 | perfil de entrenador sin rol trainer (RN-22) | Rechazo | Correcto |
| T56 | retirar el rol trainer con perfil existente | Rechazo | Correcto |
| T57 | asignar alumno a su entrenador | Aceptación | Correcto |
| T58 | entrenador asignado como su propio alumno | Rechazo | Correcto |
| T59 | alumno sin rol client | Rechazo | Correcto |
| T60 | alumno de otro gimnasio | Rechazo | Correcto |
| T61 | rutina para un alumno propio | Aceptación | Correcto |
| T62 | rutina para un cliente que no es su alumno (RN-17) | Rechazo | Correcto |
| T63 | ejercicio global en rutina | Aceptación | Correcto |
| T64 | ejercicio propio del mismo gimnasio en rutina | Aceptación | Correcto |
| T65 | ejercicio propio de otro gimnasio en rutina | Rechazo | Correcto |
| T66 | orden de ejecución repetido en la rutina | Rechazo | Correcto |
| T67 | nombre de ejercicio duplicado en el catálogo global | Rechazo | Correcto |
| T68 | mismo nombre de ejercicio en un gimnasio distinto al global | Aceptación | Correcto |
| T69 | equipo en sede propia | Aceptación | Correcto |
| T70 | equipo en sede de otro gimnasio | Rechazo | Correcto |
| T71 | evento de auditoría | Aceptación | Correcto |
| T72 | la auditoría no se modifica | Rechazo | Correcto |
| T73 | la auditoría no se borra | Rechazo | Correcto |
| T74 | vista: membresía vencida se calcula como expired | Según condición | Correcto |
| T75 | vista: membresía vigente se calcula como active | Según condición | Correcto |
| T76 | sesión con token_hash repetido | Rechazo | Retirada en la v3 |
| T77 | sesión con expiración anterior a su creación | Rechazo | Retirada en la v3 |
| T78 | membresía suspendida o cancelada sin motivo, fecha o autor | Rechazo | Por ejecutar |
| T79 | membresía cancelada con motivo, fecha y autor | Aceptación | Por ejecutar |
| T80 | pago con completed_at sin completed_by_member_id | Rechazo | Por ejecutar |
| T81 | completar un pago pendiente con fecha y autor | Aceptación | Por ejecutar |
| T82 | autor de cambio de estado de otro gimnasio | Rechazo | Por ejecutar |
| T83 | sesión del Administrador Master sin organization_id | Aceptación | Retirada en la v3 |

| N° | Prueba | Resultado esperado | Resultado |
|---|---|---|---|
| T84 | entrenamiento de una rutina propia y activa | Aceptación | Correcto (esquema parcial) |
| T85 | entrenamiento de una rutina de otro cliente (RN-41) | Rechazo | Correcto (esquema parcial) |
| T86 | entrenamiento de una rutina archivada (RN-41) | Rechazo | Correcto (esquema parcial) |
| T87 | entrenamiento con fecha futura (RN-41) | Rechazo | Correcto (esquema parcial) |
| T88 | entrenamiento con una afiliación de otro gimnasio | Rechazo | Correcto (esquema parcial) |
| T89 | ejercicio completado que pertenece a la rutina | Aceptación | Correcto (esquema parcial) |
| T90 | ejercicio completado de otra rutina (RN-41) | Rechazo | Correcto (esquema parcial) |
| T91 | peso corporal válido | Aceptación | Correcto (esquema parcial) |
| T92 | peso corporal fuera de 20 a 500 kg | Rechazo | Correcto (esquema parcial) |
| T93 | segundo registro de peso el mismo día (RN-43) | Rechazo | Correcto (esquema parcial) |
| T94 | meta semanal de 8 entrenamientos | Rechazo | Correcto (esquema parcial) |
| T95 | calorías aproximadas de una rutina iguales a cero | Rechazo | Correcto (esquema parcial) |

## 8.1 Prueba de concurrencia

Se probó con dos sesiones de psql simultáneas sobre una clase con un único cupo libre: la sesión A insertó la reserva y confirmó la transacción (INSERT 0 1) y la sesión B, que intentaba reservar el mismo cupo, recibió «RN-15: la clase no tiene cupo» y no insertó nada. Resultado: exactamente una reserva confirmada. **Alcance de la prueba:** dos sesiones, no las 100 solicitudes simultáneas de RNF-09; esa ampliación debe hacerse desde el backend.

## 8.2 Qué no se probó

- Las reglas con mecanismo en el backend (sección 6): dependen de código que aún no existe.
- Rendimiento con volumen (RNF-02) y restauración de copias (RNF-11).
- Row-Level Security: es una propuesta (sección 9), no está implementada.

- Las pruebas T78 a T82 de la v2 y la migración sobre el esquema completo; la migración solo se probó sobre tablas simuladas.
- Las pruebas T84 a T95 sobre el esquema completo (solo se ejecutaron sobre un esquema parcial) y las reglas RN-42 y RN-43, que dependen del código del backend.

## 9. Defensa en profundidad del aislamiento (propuesta, no implementada)

Hoy el aislamiento entre establecimientos se logra con claves foráneas compuestas (integridad entre registros) y con el filtro de establecimiento que el backend toma de la sesión. Como capa adicional, PostgreSQL permite Row-Level Security (RLS): el backend establecería el establecimiento de la sesión con SET LOCAL app.org_id y cada tabla con organization_id aplicaría una política que solo deje ver las filas de ese establecimiento. Ejemplo (no probado):

```
ALTER TABLE memberships ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON memberships
  USING (organization_id = current_setting('app.org_id')::bigint);
```

No se incluye en el MVP porque exige que toda conexión fije el contexto, que el rol de aplicación no sea propietario de las tablas, y pruebas específicas; queda como mejora futura. Si se adopta, las tablas globales (users, health_insurance_providers) y las que admiten organization_id nulo (exercises, audit_events) necesitan políticas distintas.

## 10. Sobre el componente opcional (CDC / almacenamiento columnar)

El enunciado de la etapa incluye un componente opcional de captura de cambios de datos (CDC) sobre una entidad transaccional, persistiendo cada operación Create/Update/Delete como un archivo Parquet organizado por fecha (/aaaa/mm/dd/{create|update|delete}/cedula.parquet), más un script de reportería en Python. No se desarrolla en este documento por ser explícitamente opcional. Si el equipo lo implementa, users es la elección más natural porque ya tiene document_number (cédula), como pide el patrón de nombres de archivo. Decisión del equipo (D-25): queda fuera del alcance activo y solo se reactiva si el docente lo exige.

*Este documento es la base directa de la Etapa 5 (Arquitectura).*